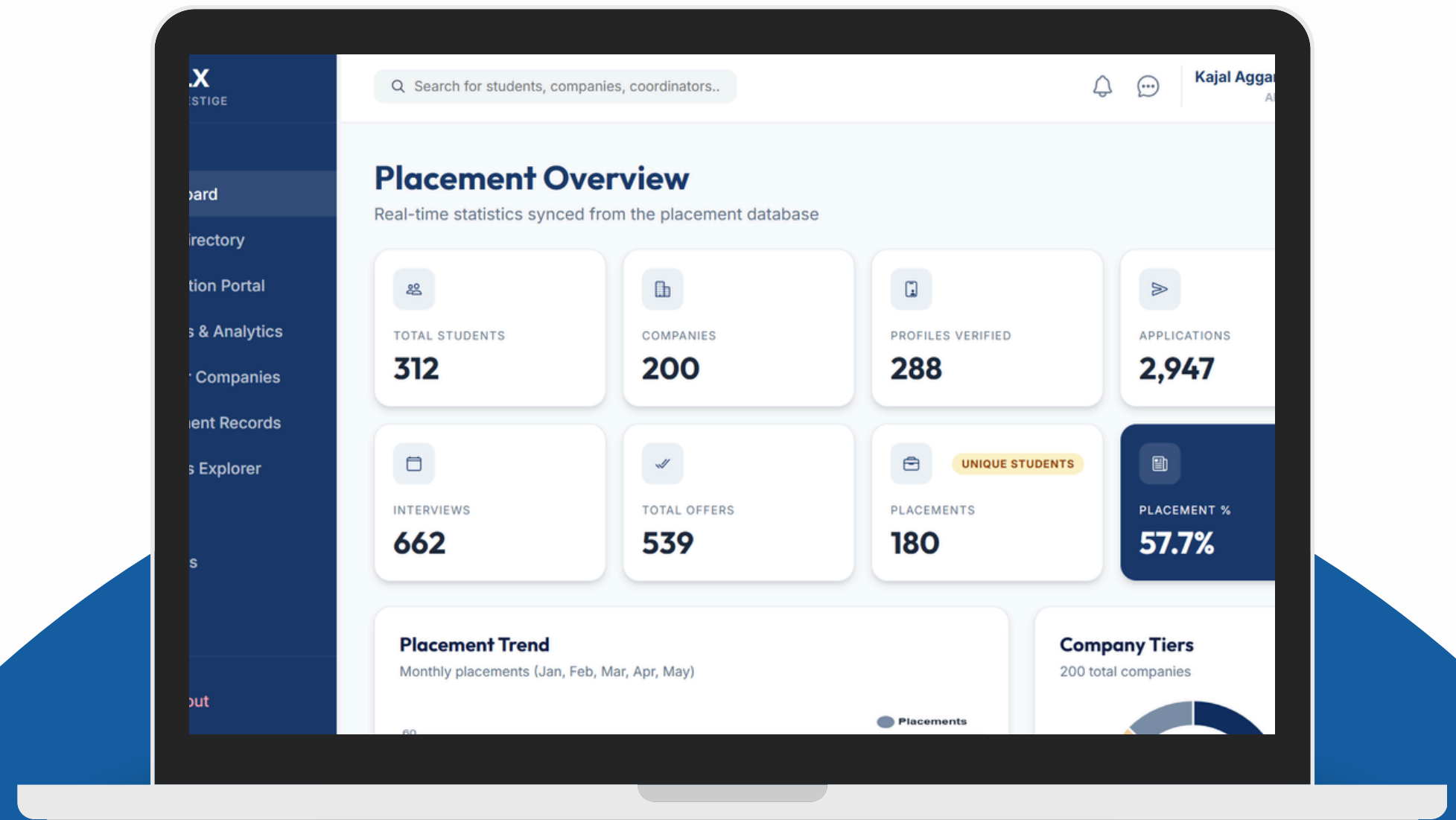


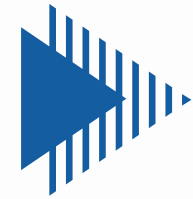
PORTALX

STUDENT PLACEMENT DATABASE MANAGEMENT SYSTEM

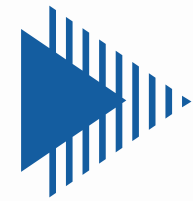
By : Group 8



Our Team



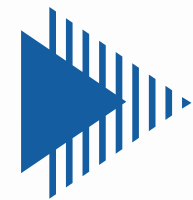
Riddhi Sharma



Manya Awasthi



Mehak Kumar

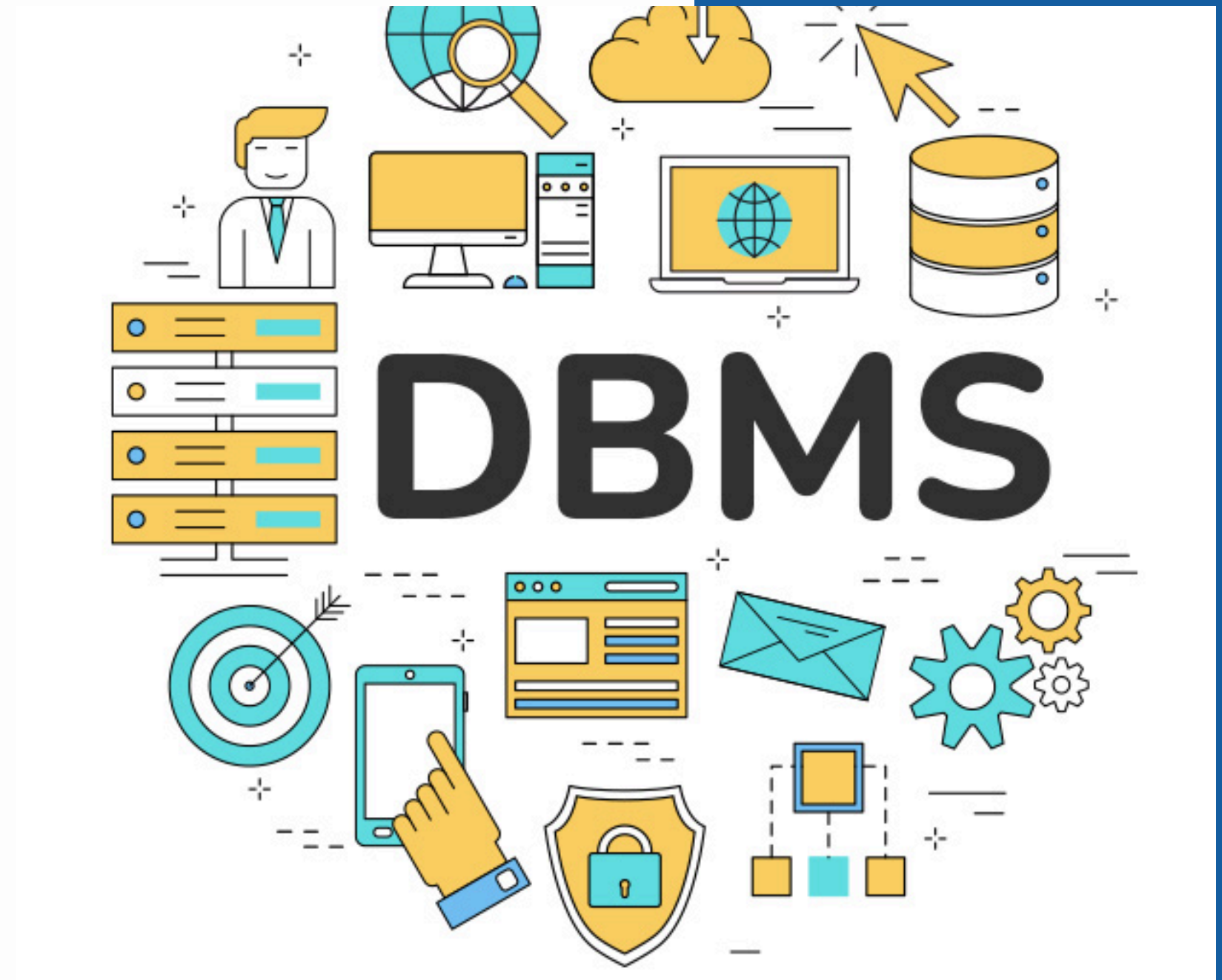


Kaavish Babbar



Table of Contents

▶▶▶ Problem Statement	03
▶▶▶ Proposed Solution	04
▶▶▶ Key Objectives	05
▶▶▶ System Architecture	06
▶▶▶ ER Overview	07
▶▶▶ Normalization	10
▶▶▶ Advanced DBMS	13
▶▶▶ Locking & Concurrency Control	18
▶▶▶ Challenges & Solutions	20
▶▶▶ Security & Future Scope	21
▶▶▶ Concurrency	24



A high-angle, top-down view of an office environment. Several people are seated at white desks, working on laptops. The desks are equipped with various office supplies like pens, paper, and water bottles. The office has a modern, open-plan feel with grey cubicle walls. The lighting is bright and even.

Problem Statement

Traditional placement processes are inefficient due to manual spreadsheet management, causing data inconsistencies and duplication. Students lack transparency, and administrators miss timely insights leading to bottlenecks and reduced placement success.



Proposed Solution

We engineered a unified, normalized relational database that centralizes all placement data. By applying rigorous database normalization (1NF/2NF/3NF), we eliminated redundancy and anomalies. Automated business logic through triggers and stored procedures replaces manual intervention. The full-stack architecture (Vite frontend + Node.js backend + MySQL database) provides seamless user experience while maintaining data integrity.

Key Objectives

- Eliminate data redundancy through normalization (1NF, 2NF, 3NF)
- Ensure data consistency through ACID compliance and triggers
- Provide secure, role-based access control
- Automate placement workflows and resume screening
- Generate actionable institutional analytics



SYSTEM ARCHITECTURE

Modern database systems require separation of concerns for scalability, maintainability, and security:

Presentation Layer (Vite Frontend)

Role-based dashboards for students, coordinators, and admins with customized features and data access.

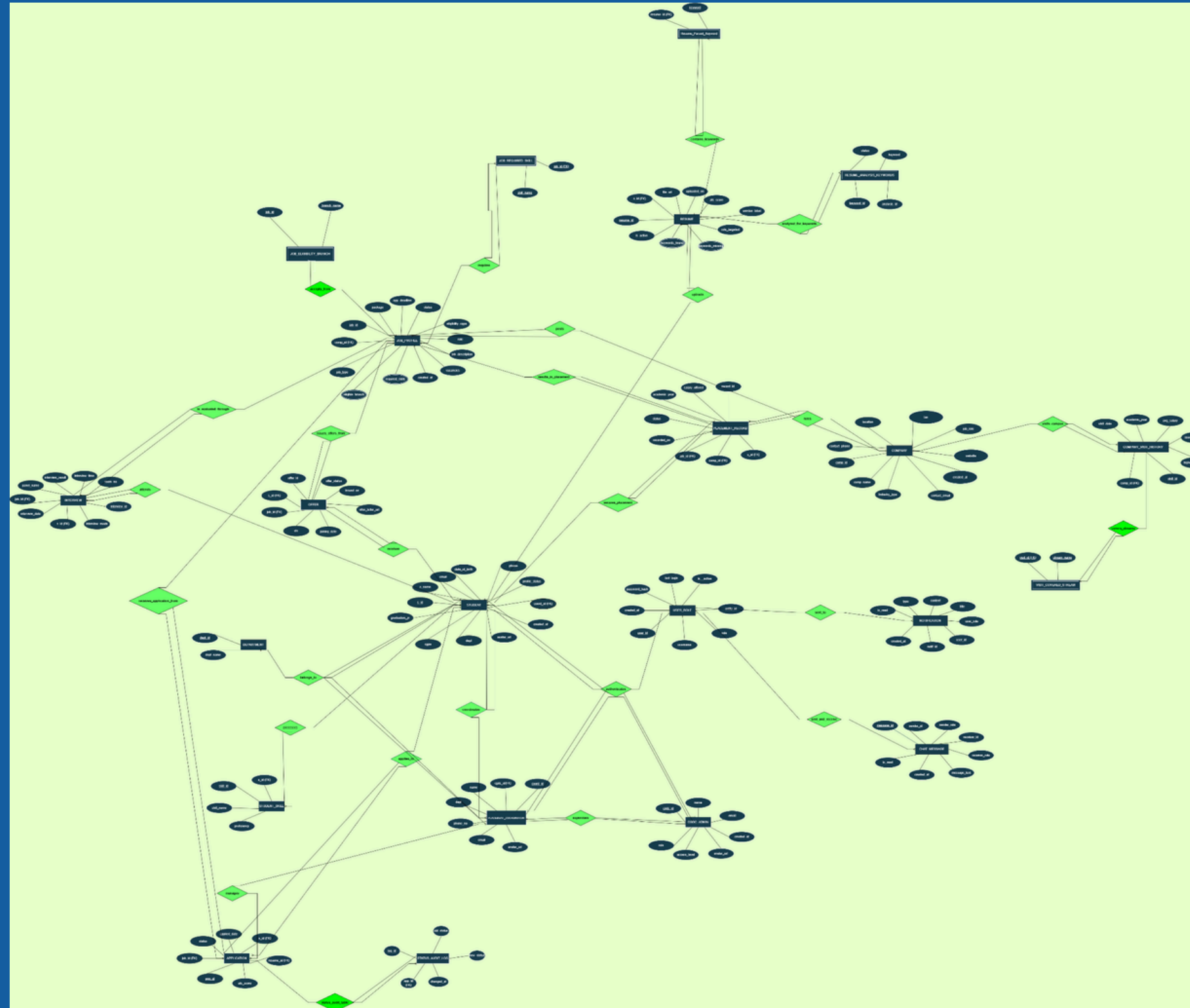
Application Layer (Node.js/Express)

Middleware layer manages business logic, file uploads, and authentication ensuring secure access without direct database exposure.

Data Layer (MySQL Database)

Database-level validation enforces constraints at the source—ensuring reliable data integrity across all access paths.



[illegible]

Entity Classification in ER Design

Master Data (8)

- STUDENT, COMPANY, DEPARTMENT, CGDC_ADMIN, PLACEMENT_COORDINATOR, JOB_PROFILE, USER_ROLE, SKILL_MASTER
- Core entities that exist independently
- Example: COMPANY exists even without JOB_PROFILE (1:N relationship)

Transactional Data (3)

- APPLICATION, INTERVIEW, OFFER
- Represent interactions between students and opportunities
- Dependent on parent entities (weak entities concept)

Entity Classification in ER Design

Historical & Audit Data (5)

- PLACEMENT_RECORD, COMPANY_VISIT_HISTORY, STATUS_AUDIT_LOG, NOTIFICATION, RESUME
- Maintain history for tracking and analysis
- Ensures past records are preserved separately from current data

Normalized Data (6)

- JOB_REQUIRED_SKILL, JOB_ELIGIBILITY_BRANCH, RESUME_PARSED_KEYWORD, STUDENT_SKILL, VISIT_COVERED_STREAM, CHAT_MESSAGE
- Convert multi-valued attributes into atomic form (1NF)
- Enable efficient searching, filtering, and indexing

NORMALIZATION & SCHEMA DESIGN

FIRST NORMAL FORM (1NF)

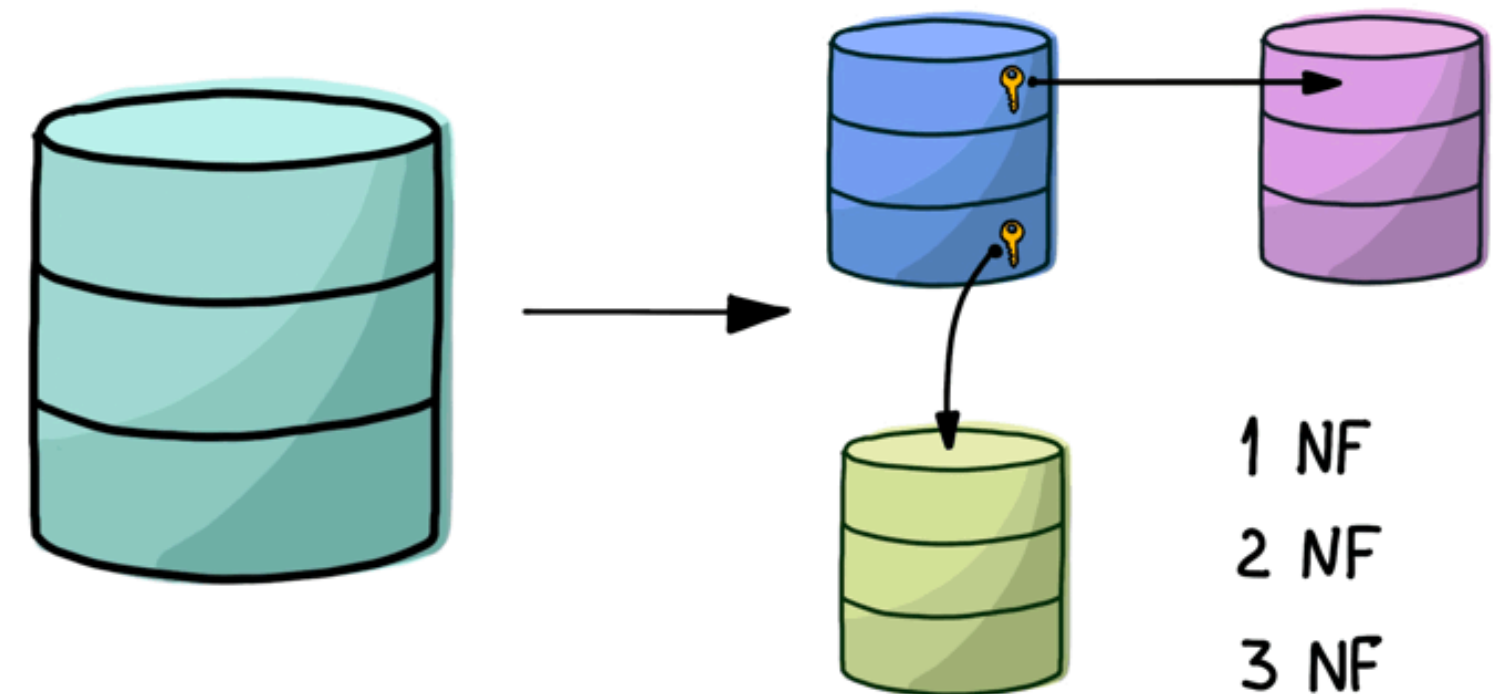
Problem (Violates 1NF):

- JOB_PROFILE.required_skills = "Python,Java,SQL,Git" (comma-separated)
- Issue:
 1. Cannot search individual skills efficiently
 2. Cannot index
 3. String pattern matching is slow

Applied to Project:

All 22 entities fully 3NF compliant

Zero transitive dependencies identified



NORMALIZATION & SCHEMA DESIGN

SECOND NORMAL FORM (2NF)

Problem (Violates 2NF):

STUDENT_SKILL Table

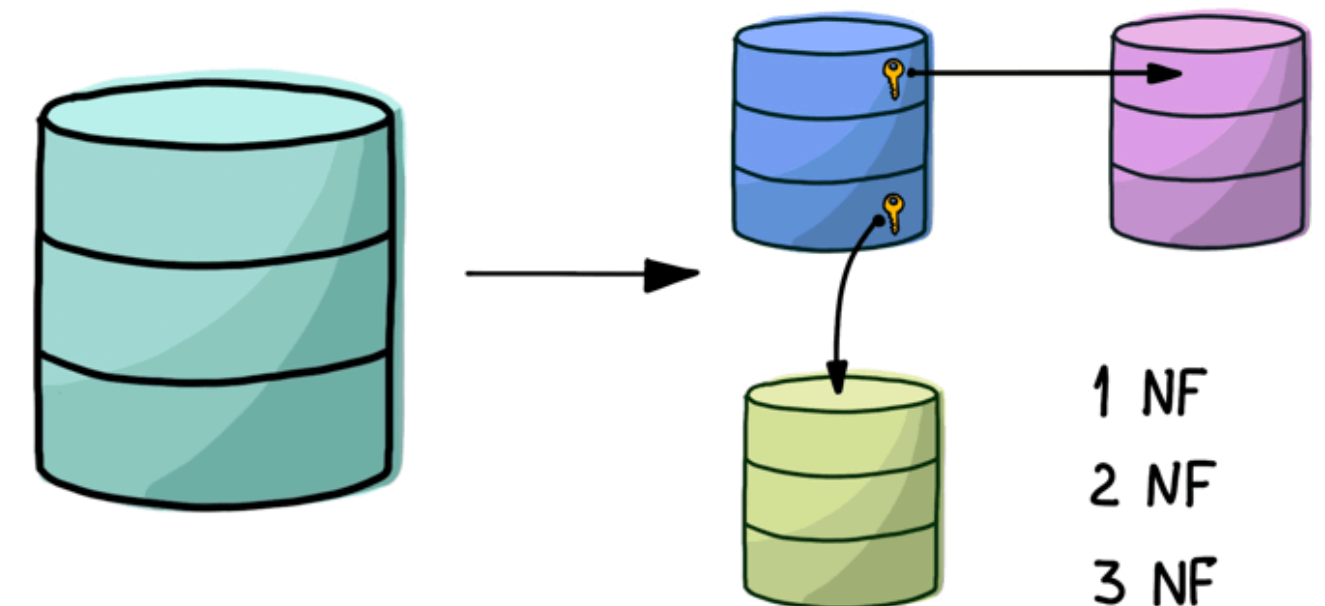
Primary Key: student_id + skill_id

student_id	skill_id	proficiency	skill_name	skill_category
1	101	Advanced	Python	Programming
2	101	Intermediate	Python	Programming

Applied to Project:

All composite keys checked for 2NF compliance

Bridge tables reference master entities via FK only



NORMALIZATION & SCHEMA DESIGN

THIRD NORMAL FORM (3NF)

Problem (Violates 3NF):

STUDENT Table (Violates 3NF)

Primary Key: s_id

s_id	s_name	dept_id	dept_name	dept_hod	coord_id	coord_name
1	Rajesh	1	CS	Dr Smith	101	Ms Johnson
2	Sneha	1	CS	Dr Smith	101	Ms Johnson
3	Priya	2	ME	Prof B	102	Mr Davis

Solution (1NF Compliant):

- JOB_REQUIRED_SKILL table (bridge table)
- One row per skill:
 - 1.job_id=1, skill_name="Python"
 - 2.job_id=1, skill_name="Java"
 - 3.job_id=1, skill_name="SQL"

INDEXING STRATEGY FOR PERFORMANCE

Without: Full table scan (all 100,000 rows checked every query)

With: Jump directly to relevant rows (hash/B-tree lookup)

Indexing Strategy:

1. Primary Key Index

Applied on all tables (s_id, comp_id, job_id)

Enables fastest data access and joins

2. Foreign Key Index

On dept_id, coord_id, s_id, job_id

Optimizes JOIN operations

3. Search Index (WHERE filters)

cgpa, profile_status, job status

Speeds up filtering queries

heavy placement systems



VIEWS

Benefits:

1. Query Simplicity

Without: Coordinators write 10-line complex query

With: `SELECT * FROM vw_student_placement_status;`

2. Data Security

Without: Grant access to STUDENT table (reveals everything)

With: Grant access only to view (show only necessary columns)

→ Coordinators cannot see passwords, personal notes

3. Consistency

All dashboards use same calculation (no duplicate logic)

Change logic once in view, all dashboards updated

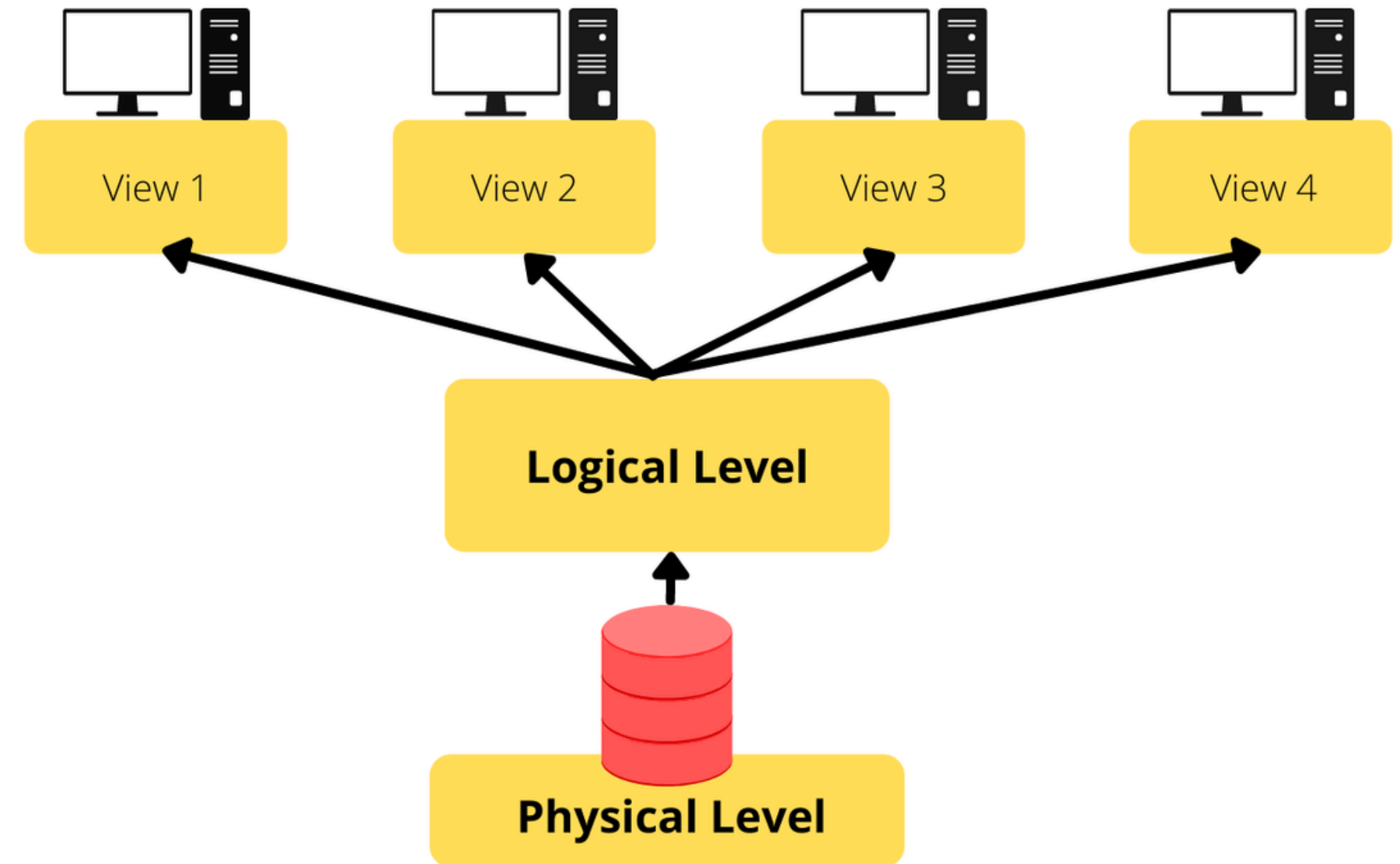
Four Views Implemented:

`vw_dashboard_stats` (system-wide placement summary)

`vw_placement_analytics` (company hiring statistics)

`vw_student_skills_summary` (student capabilities)

`vw_company_stats` (company performance metrics)



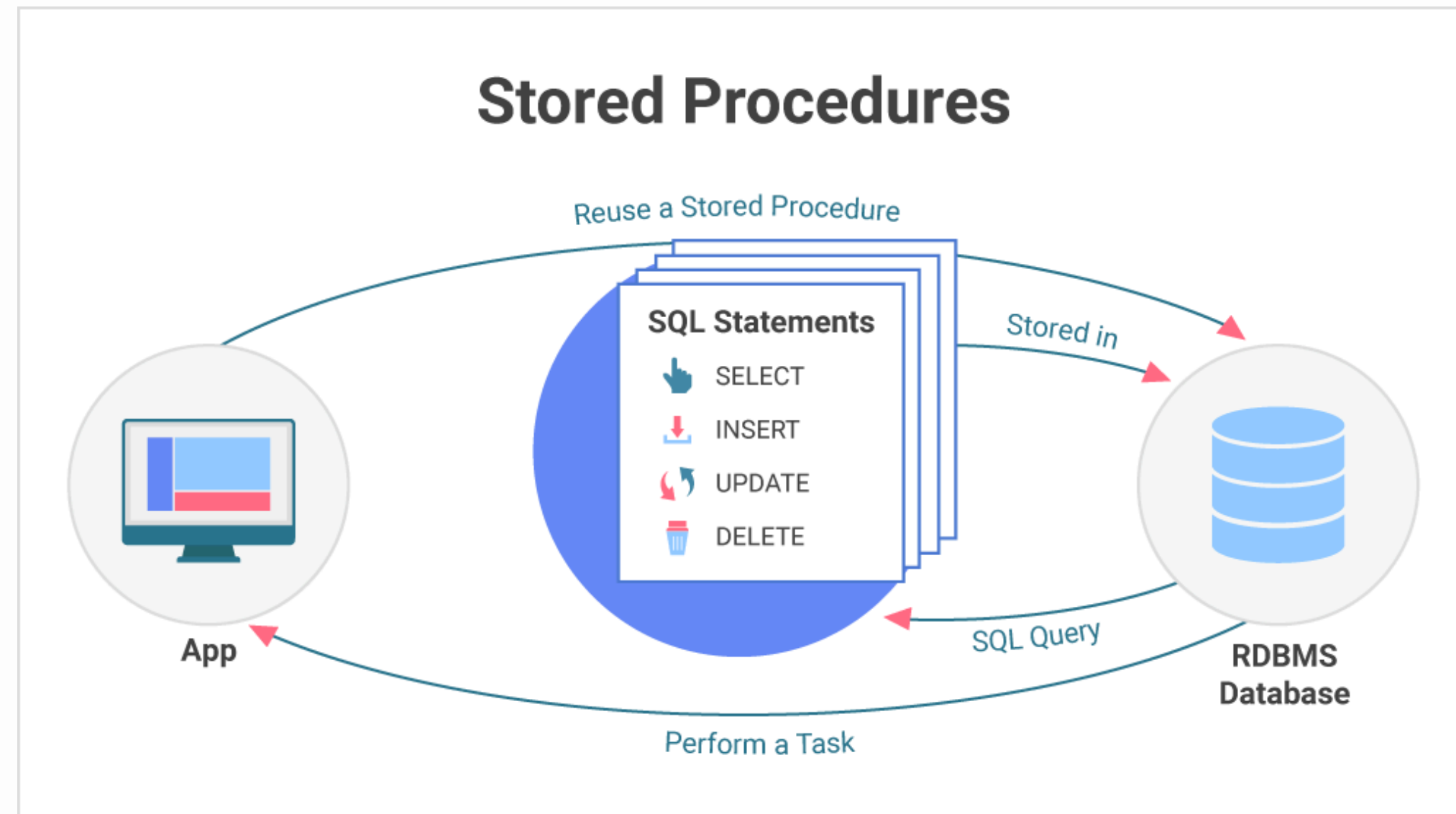
STORED PROCEDURES

Benefits:

Procedures Implemented:

- sp_accept_offer (multi-step offer acceptance)
- sp_get_company_stats (complex aggregation for analytics)
- sp_schedule_interview (lock + insert atomic)
- sp_withdraw_application (cascade deletes with audit)

Benefit: Database enforces correctness, not application code



JOINS

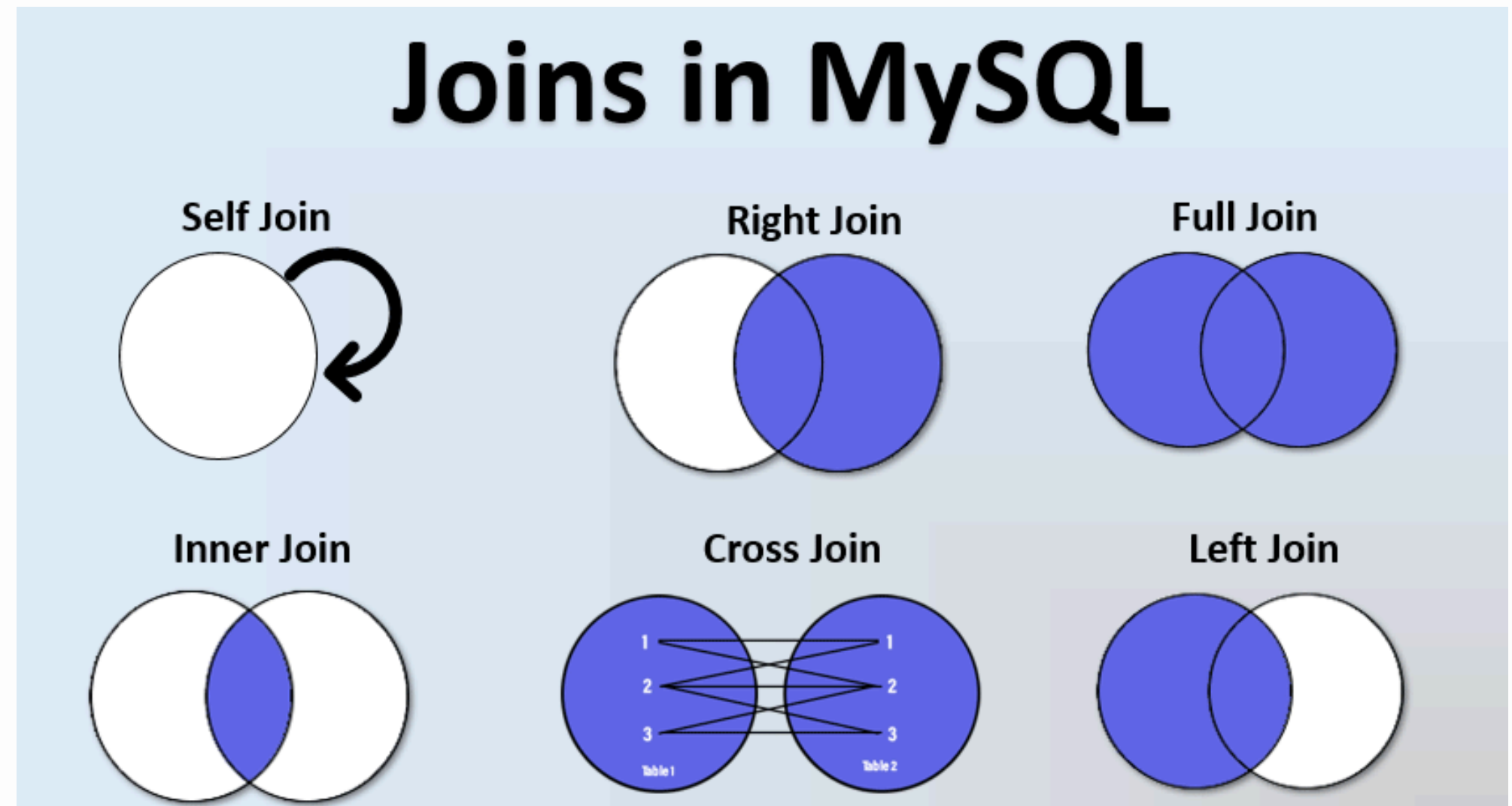
Join Usage in Project:

Dashboard: 8-table LEFT JOIN for placement summary

Analytics: 5-table INNER JOIN for company statistics

Student Profile: 6-table LEFT JOIN for skill aggregation

All JOINS use indexed foreign keys for optimization



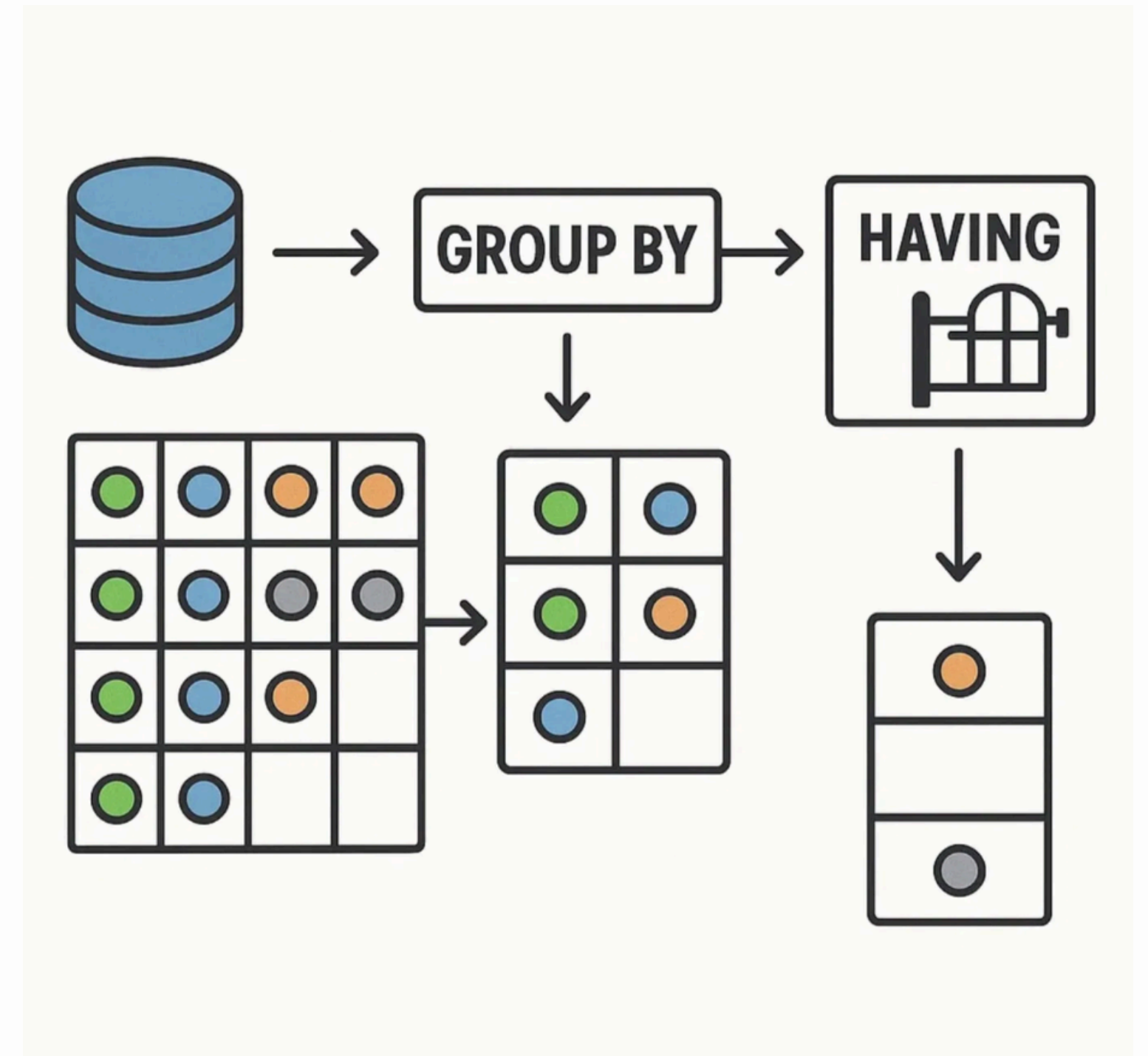
AGGREGATE FUNCTIONS & GROUP BY + HAVING

Used in Project:

Dashboard aggregations (students per status, salary ranges)

Analytics reports (company performance, department comparisons)

Query optimization (filtering early at database level)



LOCKING & CONCURRENCY CONTROL

- Problem: Race conditions cause overbooking (multiple users update same data)
- Solution: Pessimistic locking (SELECT...FOR UPDATE) ensures one transaction at a time
- Working: Row is locked → others wait → executed sequentially → no conflict

Job Offers

Prevent multiple students
taking last vacancy

Coordinator reassignment

Bulk updates done
atomically

Interview Scheduling

Avoid double-
booking of slots

TRADE OFF : Slight delay due to locking, but guarantees data integrity and consistency

PERFORMANCE & OPTIMIZATION RESULTS

Before vs After

- Dashboard: 12s → 0.3s (40x faster)
- Search: 8s → 0.15s (53x faster)
- Scalability: 10K → 1M+ users

Key Optimizations

- Indexing (PK, FK, composite) → faster queries
- Query tuning → efficient joins & filters
- Normalized DB design → reduced redundancy
- Connection pooling → better resource usage
- Caching → reduced database load

Performance Metrics

- Avg query: <100ms
- Complex joins: ~250ms
- Dashboard: ~300ms
- Search: ~150ms

System Capacity

- Handles 1000+ concurrent users
- 50K+ daily queries smoothly
- Scales linearly with growth

IMPLEMENTATION CHALLENGES & SOLUTIONS

- Race condition: Row-level locking → prevents multiple offer acceptance
- Interview conflicts: Composite locking → avoids double-booking
- Orphan data: Cascading deletes → ensures clean data removal
- Audit lag: Async logging → minimizes performance impact
- Data inconsistency: Transactions & procedures → ensures atomic updates
- Access control: Role-based views → restricts department-level data
- Slow skill search: Full-text indexing → enables fast resume filtering

Security & Role Based Access Control

Roles

- Student: Own data, apply jobs | No access to others
- Coordinator: Dept students + interviews | Limited scope
- Admin: Full system access

Access Control

- Views only
(vw_student_eligible_jobs, etc.)
- No direct table access → restricted exposure

Permissions

- Student → SELECT
- Coordinator → SELECT + UPDATE
- Admin → Full (CRUD)

Security & Role Based Access Control

Security

- Passwords: bcrypt hashed
- Auth: Node.js
- SQL Injection: Parameterized queries

Audit

- STATUS_AUDIT_LOG + triggers
- Tracks all changes (tamper-resistant)

Data Protection

- Encrypted sensitive data
- Secure uploads
- HTTPS API

Future Enhancements

01

Database replication + Redis caching
for scalability & performance

02

ML-based resume parsing for
automated and faster hiring

03

Predictive analytics for placement
insights and decision-making

04

Real-time notifications + mobile app
for better accessibility



Conclusion

The system transforms manual placement operations into a fully normalized, automated, and scalable database solution, eliminating redundancy through 1NF, 2NF, and 3NF while significantly improving performance with optimized indexing and queries . It ensures data consistency and reliability using ACID transactions, triggers, and constraints, while views and role-based access control enhance security. By automating workflows and enabling real-time analytics, the system reduces manual effort, supports faster decision-making, and provides a robust, industry-ready solution capable of handling large-scale placement data efficiently.

THANK YOU!

